Name _____

CSCI 332, Fall 2026

Exam 1 Collected Problems

This practice exam collects the problems on the following topics from the Fall 2025 exams, practice exams, and final exam:

1. Stable Matching
2. Algorithm Analysis
3. Recursion
4. 1D Dynamic Programming

Problems are grouped by topic and labeled with their source exam. Original point values, where provided, are retained for reference; this collection is not a single 100-point exam. Repeated problems and variants from the source exams are included.

If you need more space than what is given, use additional scratch paper.

Good luck!

Section 1 (Stable Matching)

From Fall 2025 Exam 1

1. (7 points) The input to the *stable matching problem* is a set of n men, a set of n women, and preference lists ranking each man for each woman and each woman for each man. Describe the desired output as precisely as you can.

   A matching for the following preference lists for 4 men and 4 women is given by the capitalized men and women.

   $$\begin{array}{ll} m_1\colon w_1, w_4, w_3, W_2 & w_1\colon m_4, m_3, M_2, m_1 \\ m_2\colon W_1, w_3, w_2, w_4 & w_2\colon m_4, m_2, M_1, m_3 \\ m_3\colon w_4, w_1, W_3, w_2 & w_3\colon m_1, M_3, m_2, m_4 \\ m_4\colon W_4, w_3, w_2, w_1 & w_4\colon m_3, m_1, m_2, M_4 \end{array}$$

2. (5 points) Is this matching stable? Yes or no

3. (5 points) How do you know?

4. (8 points) This is the same preference list for 4 men and 4 women given above.

   $$\begin{array}{ll} m_1\colon w_1, w_4, w_3, w_2 & w_1\colon m_4, m_3, m_2, m_1 \\ m_2\colon w_1, w_3, w_2, w_4 & w_2\colon m_4, m_2, m_1, m_3 \\ m_3\colon w_4, w_1, w_3, w_2 & w_3\colon m_1, m_3, m_2, m_4 \\ m_4\colon w_4, w_3, w_2, w_1 & w_4\colon m_3, m_1, m_2, m_4 \end{array}$$

   What is the outcome of the Gale-Shapley algorithm on this input? Assume that men propose to women.

5. (5 points) True or false? In every instance of the Stable Matching Problem, there is a stable matching containing a pair (m, w) such that m is ranked first on the preference list of w and w is ranked first on the preference list of m.

From Fall 2025 Exam 1 Practice 1

6. (5 points) The input to the *stable matching problem* is a set of n men, a set of n women, and preference lists ranking each man for each woman and each woman for each man. Describe the desired output as precisely as you can.

7. A matching for the following preference lists for 4 men and 4 women is given by the capitalized men and women.

$$\begin{array}{ll} m_1\colon w_3, W_1, w_2, w_4 & w_1\colon m_4, m_3, m_2, M_1 \\ m_2\colon W_4, w_1, w_2, w_3 & w_2\colon M_4, m_3, m_2, m_1 \\ m_3\colon w_2, w_1, W_3, w_4 & w_3\colon m_1, m_2, m_4, M_3 \\ m_4\colon w_1, W_2, w_3, w_4 & w_4\colon m_1, m_3, m_4, M_2 \end{array}$$

8. (3 points) Is this matching stable? Yes or no

9. (2 points) How do you know?

10. (5 points) This is the same preference list for 4 men and 4 women given above.

$$\begin{array}{ll} m_1\colon w_3, w_1, w_2, w_4 & w_1\colon m_4, m_3, m_2, m_1 \\ m_2\colon w_4, w_1, w_2, w_3 & w_2\colon m_4, m_3, m_2, m_1 \\ m_3\colon w_2, w_1, w_3, w_4 & w_3\colon m_1, m_2, m_4, m_3 \\ m_4\colon w_3, w_2, w_1, w_4 & w_4\colon m_1, m_3, m_4, m_2 \end{array}$$

What is the outcome of the Gale-Shapley algorithm on this input? Assume that men propose to women.

11. (5 points) True or false? In every instance of the Stable Matching Problem, there is a stable matching containing a pair (m, w) such that m is ranked first on the preference list of w and w is ranked first on the preference list of m.

From Fall 2025 Exam 1 Practice 2

Recall the Gale-Shapley Algorithm for solving the stable matching problem:

```
Gale-Shapley(preference lists for n hospitals and n students)
  Let M be an empty matching
  While some hospital h is umatched and hasn't proposed to every student:
      Let s be the first student on h's list to whom h has not yet proposed
      If s is unmatched:
          Add (h,s) to M
      Else s is matched to h':
          If s prefers h to h':
              Remove (h',s) from M
              Add (h,s) to M
          Else s prefers h' to h:
              Do nothing
  Return M
```

12. (10 points) For each of the following claims about the Gale-Shapley algorithm on any input, mark whether they are true or false.

    (a) Once a hospital becomes matched, it never becomes unmatched. T or F

    (b) Once a student becomes matched, it never becomes unmatched. T or F

13. (5 points) Describe a best-case input for the Gale-Shapley algorithm.

14. (10 points) Give the smallest, simplest function $f(n)$ that you can such that the the number of iterations of the while loop in the Gale-Shapley algorithm is $O(f(n))$ on any input of n hospitals and n students.

15. (5 points) Write the name or definition of the following set symbols:

    (a) $\subseteq$:

    (b) $\cup$:

    (c) $\cap$:

    (d) $\mathscr{P}$:

    (e) $\in$:

From Fall 2025 Final Exam

16. Is $\{(h_1, s_4), (h_2, s_1), (h_3, s_3), (h_4, s_2)\}$ a stable matching for the given preference lists?

Hospital	Preferences	Student	Preferences
h_1	s_2, s_3, s_4, s_1	s_1	h_3, h_1, h_2, h_4
h_2	s_1, s_3, s_2, s_4	s_2	h_2, h_3, h_4, h_1
h_3	s_3, s_4, s_2, s_1	s_3	h_3, h_2, h_4, h_1
h_4	s_1, s_3, s_2, s_4	s_4	h_4, h_1, h_3, h_2

Select A for yes and B for no.

17. Given the matching $\{(h_4, s_1), (h_1, s_3), (h_3, s_4), (h_2, s_2)\}$ and the preference lists below, select all unstable pairs from the list.

Hospital	Preferences	Student	Preferences
h_1	s_1, s_2, s_3, s_4	s_1	h_3, h_2, h_1, h_4
h_2	s_1, s_2, s_4, s_3	s_2	h_1, h_2, h_3, h_4
h_3	s_2, s_1, s_3, s_4	s_3	h_1, h_3, h_4, h_2
h_4	s_4, s_2, s_3, s_1	s_4	h_3, h_4, h_2, h_1

A. (h_4, s_1)

B. (h_2, s_4)

C. (h_2, s_1)

D. (h_3, s_3)

E. (h_1, s_2)

The following questions refer to the Gale-Shapley algorithm for stable matching.

```
Gale-Shapley(preference lists for n hospitals and n students)
  Let M be an empty matching
  While some hospital h is umatched and hasn't proposed to every student:
      Let s be the first student on h's list to whom h has not yet proposed
      If s is unmatched:
          Add (h, s) to M
      Else s is matched to h':
          If s prefers h to h':
              Remove (h', s) from M
              Add (h, s) to M
          Else s prefers h' to h:
              Do nothing
  Return M
```

18. Does the Gale-Shapley algorithm always terminate with a stable matching? Select A for yes and B for no.

19. Are there inputs for which the Gale-Shapley algorithm runs in $O(n)$ time? Select A for yes and B for no.

20. Are there inputs for which the Gale-Shapley algorithm runs in $O(n^2)$ time? Select A for yes and B for no.

21. Are there inputs for which the Gale-Shapley algorithm runs in $O(n^3)$ time? Select A for yes and B for no.

22. What is the stable matching returned by the Gale-Shapley algorithm on the following preference list?

Hospital	**Preferences**	**Student**	**Preferences**
h_1	s_1, s_2, s_3, s_4	s_1	h_3, h_2, h_1, h_4
h_2	s_1, s_2, s_4, s_3	s_2	h_1, h_2, h_3, h_4
h_3	s_2, s_1, s_3, s_4	s_3	h_1, h_3, h_4, h_2
h_4	s_4, s_2, s_3, s_1	s_4	h_3, h_4, h_2, h_1

A. $\{(h_1, s_1), (h_2, s_1), (h_3, s_2), (h_4, s_4)\}$

B. $\{(h_1, s_1), (h_2, s_2), (h_3, s_3), (h_4, s_4)\}$

C. $\{(h_1, s_4), (h_2, s_3), (h_3, s_1), (h_4, s_2)\}$

D. $\{(h_1, s_2), (h_2, s_3), (h_3, s_1), (h_4, s_4)\}$

E. $\{(h_1, s_3), (h_2, s_1), (h_3, s_4), (h_4, s_2)\}$

Section 2 (Algorithm Analysis)

From Fall 2025 Exam 1

23. (10 points) Suppose you have algorithms with the following runtimes. (Assume these are exact running times as a function of the input size n, not asymptotic running times.) How much slower do these algorithms get when you double the input size? (You can find this by dividing the runtime on $2n$ by the runtime on n). You should simplify your answer as much as possible, including evaluating logarithms when possible.

    (a) $3n^2$

    (b) $\log_2 n$

    (c) 3^n

24. (5 points) Give a function $f(n)$ such that the worst-case runtime of the following algorithm is $\Theta(f(n))$.

```
Algorithm-1(array A of length n):
    Result = 0
    For each i in 1 to n:
        For each j in 1 to i:
            Set Result to A[j] times Result
    Return Result
```

25. (10 points) Order the following functions from asymptotically smallest to asymptotically largest. If two functions are asymptotically equal (one is Θ of the other), indicate this with an equals sign. For example, if the functions were n, n^2, and $3n+4$, the answer would be $n = 3n+4 < n^2$.

    - $n \log n^3$
    - $n \log n^2$
    - $\sqrt{n}$
    - $n + \log n$
    - $n^{\log_3 2}$
    - $100^{100!}$
    - $\log_2 n$
    - 2^n
    - 3^n

26. (5 points) Suppose you know that an algorithm has a worst-case runtime that is $O(n^2 \log n)$. For each of the following, decide whether it is definitely true, definitely false, or could be true or false and circle your choice.

    - The algorithm's worst-case runtime is $\Omega(n^2)$. T, F, T or F
    - The algorithm's worst-case runtime is $\Omega(n^3)$. T, F, T or F
    - The algorithm's best-case runtime is $O(n^2)$. T, F, T or F
    - The algorithm's worst-case runtime is $O(n^3)$. T, F, T or F
    - The algorithm's worst-case runtime is $\Theta(n^2 \log n)$. T, F, T or F

From Fall 2025 Exam 1 Practice 1

27. (10 points) For each of the following statements, circle T if it is true and F if it is false.

    - 2^n is $O(n!)$: T or F
    - n^3 is $\Omega(n^2)$: T or F
    - $n \log n$ is $\Theta(n^2)$: T or F
    - There is an algorithm with worst-case runtime that is $\Omega(n^2)$ and best-case runtime that is $\Omega(n \log n)$: T or F
    - There is an algorithm with worst-case runtime that is $O(n^2)$ and best-case runtime that is $\Theta(n^3)$: T or F

28. (5 points) Give a function $f(n)$ such that the worst-case runtime of the following algorithm is $\Theta(f(n))$.

```
Algorithm-1(array A of length n):
    Result = 0
    For every third i in 1 to n:
        For j in 1 to i:
            Set Result to A[j] times Result
    Return Result
```

29. (5 points) For the same algorithm above, give a function $g(n)$ such that the best-case runtime of the algorithm is $\Theta(g(n))$.

From Fall 2025 Exam 1 Practice 2

30. (5 points) Evaluate the following:

    (a) $\log_2(16) =$

    (b) $2^{\log_2(123456789)} =$

31. (15 points) Recall the definition of big O: a function $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that for every $n \geq n_0$, $f(n) \leq cg(n)$.

    Use this definition to prove that $3n^2 + 5n + 10$ is $O(n^2)$.

32. (10 points) Give a function $f(n)$ such that the worst-case runtime of the following algorithm is $\Theta(f(n))$. Recall that $\lfloor x \rfloor$ takes the *floor* of x, meaning that it rounds down to the nearest integer.

```
Algorithm-1(array A of length n):
    Result = 0
    For i in 1 to n:
        If A[i] is even:
            For j in i + ⌊n/2⌋ to n:
                Add A[j] to Result
    Return Result
```

From Fall 2025 Final Exam

33. Recall that the definition of big O is as follows: *$f(n)$ is $O(g(n))$ if there exist positive constants c and n_0 such that for all $n >= n_0$, $f(n) \leq c \cdot g(n)$.* Which n_0 and c can be used to show that $9n^2 + n + 10$ is $O(n^3)$?

    A. $n_0 = 1$, $c = 4$
    B. $n_0 = 1$, $c = 10$
    C. $n_0 = 1$, $c = 1000$
    D. $n_0 = 10, c = 4$
    E. $n_0 = 100$, $c = 1$

34. Select all of the true statements below.

    A. $\log(100 \cdot n^{5n})$ is $\Theta(n \log n)$
    B. $\log(100 \cdot n^{5n})$ is $\Theta(n^n)$
    C. $\log(100 \cdot n^{5n})$ is $\Theta(5^n)$
    D. $\log(100 \cdot n^{5n})$ is $\Theta(n^n \log n)$
    E. $\log(100 \cdot n^{5n})$ is $\Theta(n^5 \log 100)$

35. For which of the following is it true that $f_1(n)$ is $O(f_2(n))$ and $f_2(n)$ is $O(f_3(n))$? (That is, the functions are in increasing order of asymptotic growth, but two or more adjacent functions may have the same asymptotic growth rate.) There may be more than one correct answer; select all that apply.

    A. $f_1 = \frac{n}{10^{10}}$, $f_2 = n^{0.6}$, $f_3 = \log n$
    B. $f_1 = \log n$, $f_2 = \frac{n}{10^{10}}$, $f_3 = n^{0.6}$
    C. $f_1 = n^{0.6}$, $f_2 = \log n$, $f_3 = \frac{n}{10^{10}}$
    D. $f_1 = \log n$, $f_2 = n^{0.6}$, $f_3 = \frac{n}{10^{10}}$
    E. $f_1 = \frac{n}{10^{10}}$, $f_2 = \log n$, $f_3 = n^{0.6}$

36. For which of the following is it true that $f_1(n)$ is $O(f_2(n))$ and $f_2(n)$ is $O(f_3(n))$? (That is, the functions are in increasing order of asymptotic growth, but two or more adjacent functions may have the same asymptotic growth rate.) There may be more than one correct answer; select all that apply.

    A. $f_1 = 4^n$, $f_2 = n!$, $f_3 = 3^n$
    B. $f_1 = 3^n$, $f_2 = 4^n$, $f_3 = n!$
    C. $f_1 = 3^n$, $f_2 = n!$, $f_3 = 4^n$
    D. $f_1 = n!$, $f_2 = 4^n$, $f_3 = 3^n$
    E. $f_1 = n!$, $f_2 = 3^n$, $f_3 = 4^n$

37. Select all functions that are $\Theta(n^2)$.

A. $5n^2 + 3n + 20$

B. $n \log n$

C. $n^2 + n \log n$

D. $0.5n^2 - 10n + 2000$

E. n^3

38. Select all functions that are $\Omega(n^2)$.

A. $5n^2 + 3n + 20$

B. $n \log n$

C. $n^2 + n \log n$

D. $0.5n^2 - 10n + 2000$

E. n^3

39. Select all functions $f(n)$ such that the worst-case runtime of Algorithm-1 below is $\Theta(f(n))$.

```
Algorithm-1(array A of length n):
    Result = 0
    For each i in 1 to n:
        For each j in 1 to i:
            Set Result to A[j] times Result
    Return Result
```

A. n

B. $\log n$

C. n^2

D. n^3

E. $n!$

Section 3 (Recursion)

From Fall 2025 Exam 2

40. (10 points) Given the following (meaningless) divide and conquer algorithm, what is the recurrence relation that describes its running time?

```
wut(integer array A of length n):
  if n ≤ 1:
      return A
  else:
      Let A_1, A_2, A_3 be the first third, middle third, and last third of A
      X_1 = wut(A_1)
      X_2 = wut(A_2)
      X_3 = wut(A_3)
      let array C be a new array of length n
      for i from 1 to n:
          for j from 1 to n:
              C[i%j] = first element of X_1 + middle element of X_2 + last element of X_3
      return C
```

$T(1) =$ _____

$T(n) =$ _____ + _____

work done by recursive calls | nonrecursive work

41. (15 points)

Consider the recurrence $T(n) = 4T(n/3) + n, T(1) = 1$.

(a) (5 points) In the left boxes, draw the first three levels of this recursion tree. In the boxes next to them, write the total work on that level.

Level 0	Sum level 0
Level 1	Sum level 1
Level 2	Sum level 2

(b) (3 points) What is the work per level in terms of level ℓ?

(c) (2 points) Is the work per level (circle one)

- increasing (work is dominated by leaves)
- decreasing (work is dominated by root)
- the same (work is the same at every level)

(d) (3 points) What is the total number of levels in the tree? (fill in the base and the argument of the logarithm)

$$\text{number of levels} = \log_____(_____)$$

(e) (2 points) What is the overall non-recursive runtime of the recurrence?

42. (10 points total) In this problem you will write a **divide and conquer** algorithm for the following problem: you are given an array A of n integers (which may be positive, negative, or zero). You know that the array is *unimodal*, meaning that there is an index i^* such that $A[1] < A[2] < \cdots < A[i^*]$ and $A[i^*] > A[i^* + 1] > \cdots > A[n]$. Your task is to find the index i^* (the index of the maximum element) in $O(\log n)$ time. Assume that the indices start at 1.

For example, if $A = [-1, 3, 3, 5, 8, 12, 4, 2]$, then the output should be 6 since $A[6] = 12$ is the maximum element.

(a) (2 points) In order to **divide** the problem, you notice that i^* must either occur in (fill in the blanks):

- _____, or
- _____

(b) (2 points) In order to achieve an $O(\log n)$ runtime, you can only make one recursive call per level of recursion, and it must take only constant time. How will you decide which of your subproblems to make the recursive call on?

(c) (6 points) Write a **recursive** algorithm to find i^* that runs in $O(\log n)$ time. To simplify pseudocode, you may assume that the length of the input array is always a power of 2, To simplify the base case(s), you may also assume that the input array has length at least 4.

Max(unimodal array A of length n):

From Fall 2025 Exam 2 Practice 1

In this section, we will think about the computational problem of where to insert a new element into a sorted array. Specifically, we will return the position *after which* to insert the element. For example, in the array $A = [5, 40, 44, 600]$, we would insert the element 41 after index 1. If we need to insert the new element before the current first element, we return -1.

43. Let $A = [-85, -45, -5, 6, 12, 18]$. After which index would we insert a new element 15? (You only get points if you give an index here, not "between x and y" or something similar.) Assume the indices start at 0.

44. Give a naive, $\Theta(n)$ worst-case time algorithm to solve the Insert problem. Remember to return the index after which to insert the new element.

NaiveInsert(sorted array A, new element t):

45. We can solve this problem using divide and conquer based on the following observation: given a sorted array A and a new element t, the index after which to insert t must either occur in (fill in the blanks):

- _____, or
- _____

The pseudocode to implement a divide-and-conquer Insert is shown below. Recall that $\lfloor x \rfloor$ takes the *floor* of x, meaning that it rounds down to the nearest integer.

```
Insert(sorted array A, new element t):
    min = 0
    max = length of A - 1
    make A and t global
    return InsertRecursive(min, max)
```

```
InsertRecursive(index min, index max):
    If min = max:
        If t < A[min]:
            Return min -1
        Else:
            Return min
    Else:
        mid = ⌊ (min + max) / 2 ⌋
        If t < A[mid]:
            return InsertRecursive(min, mid - 1)
        Else:
            return InsertRecursive(mid + 1, max)
```

46. Suppose you have the length 16 array A below, with indices labeled above the array entries:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	5	6	12	100	105	108	133	229	277	280	551	559	601	603

and new element $t = 7$ and call Insert(A, t). Draw the recursion tree for the resulting call to InsertRecursive, labeling each node with the values of min and max for that call. The first node is given for you.

InsertRecursive(0, 15)

47. Describe a worst-case input for Insert, or explain why all inputs take asymptotically the same time.

48. What is the recurrence relation for the worst-case runtime of InsertRecursive on an array of length n?

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ \underline{\qquad\qquad\qquad\qquad} & \text{if } n > 1. \end{cases}$$

49. What is the depth of the recursion tree for an input array of length n?

50. What is work done at each node?

51. Give a function $f(n)$ such that the worst-case runtime of InsertRecursive is $\Theta(f(n))$.

From Fall 2025 Exam 2 Practice 2

In this section, we will think about the computational problem of where to insert a new element into a sorted array. Specifically, we will return the position *after which* to insert the element. For example, in the array $A = [5, 40, 44, 600]$, we would insert the element 41 after index 1. If we need to insert the new element before the current first element, we return -1.

52. Let $A = [-85, -45, -5, 6, 12, 18]$. After which index would we insert a new element 15? (You only get points if you give an index here, not "between x and y" or something similar.) Assume the indices start at 0.

53. Give a naive, $\Theta(n)$ worst-case time algorithm to solve the Insert problem. Remember to return the index after which to insert the new element.

NaiveInsert(sorted array A, new element t):

54. We can solve this problem using divide and conquer based on the following observation: given a sorted array A and a new element t, the index after which to insert t must either occur in (fill in the blanks):

- _____, or
- _____

The pseudocode to implement a divide-and-conquer Insert is shown below. Recall that $\lfloor x \rfloor$ takes the *floor* of x, meaning that it rounds down to the nearest integer.

```
Insert(sorted array A, new element t):
    min = 0
    max = length of A - 1
    make A and t global
    return InsertRecursive(min, max)
```

```
InsertRecursive(index min, index max):
    If min = max:
        If t < A[min]:
            Return min -1
        Else:
            Return min
    Else:
        mid = ⌊ (min + max) / 2 ⌋
        If t < A[mid]:
            return InsertRecursive(min, mid - 1)
        Else:
            return InsertRecursive(mid + 1, max)
```

55. Suppose you have the length 16 array A below, with indices labeled above the array entries:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	5	6	12	100	105	108	133	229	277	280	551	559	601	603

and new element $t = 7$ and call Insert(A, t). Draw the recursion tree for the resulting call to InsertRecursive, labeling each node with the values of min and max for that call. The first node is given for you.

InsertRecursive(0, 15)

56. Describe a worst-case input for Insert, or explain why all inputs take asymptotically the same time.

57. What is the recurrence relation for the worst-case runtime of InsertRecursive on an array of length n?

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ \underline{\qquad\qquad\qquad\qquad} & \text{if } n > 1. \end{cases}$$

58. What is the depth of the recursion tree for an input array of length n?

59. What is work done at each node?

60. Give a function $f(n)$ such that the worst-case runtime of InsertRecursive is $\Theta(f(n))$.

From Fall 2025 Final Exam

61. Given the following (meaningless) divide and conquer algorithm, what is the recurrence relation that describes its runtime? Notice the n is the number of rows and the number of columns in the square array A, not the total number of elements.

```
hi(square array A of with n rows and n columns):
  if n = 1:
      return sum(A)
  else:
      Let A_1,A_2,A_3,A_4 be the upper left, upper right, lower left, and lower right of A
      X_1 = hi(A_1 + A_4)
      X_2 = hi(A_2 + A_3)
      C = X_1 + X_2
      return C
```

A. $T(n) = 2T(n^2/2) + \Theta(n^2)$

B. $T(n) = T(n/2) + \Theta(n^2)$

C. $T(n) = 2T(n/2) + \Theta(n^2)$

D. $T(n) = 2T(n/2) + \Theta(n)$

E. $T(n) = 2T(n-2) + \Theta(n^2)$

The next two questions refer to the recurrence relation

$$T(n) = 3T(n/4) + \Theta(n^2).$$

62. What is the work done at each level of the recursion tree for this recurrence?

A. $3^\ell + n^2$

B. $3^\ell \left(\frac{n}{4^\ell}\right)^2$

C. $\frac{4}{3}\ell^n$

D. $n^2/4 + 3^\ell$

E. $\frac{\ell \cdot n^2}{3}$

63. How many levels does the recursion tree have?

A. $\log_4 n$

B. $\log_3 n$

C. n

D. $\sqrt[3]{n}$

E. $n \log_3 4$

The following questions refer to the problem of multiplying two n-digit numbers.

64. Below is a divide and conquer algorithm to multiply two n-digit numbers x and y based on the observation that, if a and b are the first and second halves of x and c, and d are the first and second halves of y respectively, then

$$x \cdot y = 10^n \cdot a \cdot c + 10^{n/2} \cdot (a \cdot d + b \cdot c) + b \cdot d.$$

```
Split-Multiply(length n integers x, y):
    # base case
    if n = 1:
        return _____

    # recursive case
    else:
        Split x into two halves a and b
        Split y into two halves c and d
        m = n/2
        e = Split-Multiply(a, c)
        f = Split-Multiply(b, d)
        g = Split-Multiply(b, c)
        h = Split-Multiply(a, d)
        return 10^(2m) · e + 10^m · (g + h) + f
```

What should be returned in the base case (the blank above)?

A. 0

B. 1

C. $x + y$

D. $x \cdot y$

E. $x - y$

65. If $x = 1234$ and $y = 5678$, what are the values of a, b, c, and d when Split-Multiply$(x, y, 4)$ is called?

A. $a = 12$, $b = 34$, $c = 56$, $d = 78$

B. $a = 1$, $b = 234$, $c = 5$, $d = 678$

C. $a = 123$, $b = 4$, $c = 567$, $d = 8$

D. $a = 1234$, $b = 0$, $c = 5678$, $d = 0$

E. $a = 0$, $b = 1234$, $c = 0$, $d = 5678$

66. What is the runtime of Split-Multiply as a function of n, the number of digits in the input numbers x and y?

A. $\Theta(n)$

B. $\Theta(n \log n)$

C. $\Theta(n^2)$

D. $\Theta(n^{\log_2 3})$

E. $\Theta(2^n)$

Section 4 (1D Dynamic Programming)

From Fall 2025 Exam 2

67. (10 points) Given the following (meaningless) recursive definition of a subproblem, draw the recursion tree for a call to $huh(5)$ with each node labeled with the value of the argument to huh.

$$huh(i) = \begin{cases} 4 & if\ i = 1 \\ -100 & if\ i = 2 \\ 0 & if\ i = 3 \\ \min(2 + huh(i-1), huh(i-3)) & if\ i > 3 \end{cases}$$

The first node is provided for you.

huh(5)

68. (5 points) In this problem, you will need to write the English definition for the subproblem (step one of our four steps for writing dynamic programming algorithms) for the following problem.

The problem. In order to cross the river, you need to hop from stone to stone. There are a total of n stones in a line, each with height h_i. You can only jump from stone i to stone $i+1$ or stone $i+2$. To conserve energy, you want to minimize the total vertical distance that you jump (both up and down). That is, if you jump from stone i to stone j, you have covered $|h_i - h_j|$ vertical distance. What is the minimum total vertical distance needed to jump from stone 1 to stone n?

To solve this problem, we first write the English definition of the subproblems we will solve.

1. The subproblem. Let _____
_____.

69. (10 points) In this problem, you will need to write the formal recursive definition for the subproblem (step two of our four steps for writing dynamic programming algorithms) for the following problem.

The problem. In your country, there are only coins of denominations 1, 3, and 4. Given a pasitive integer n, what is the fewest number of coins you can use to represent n?

To solve this problem, we first write the English definition of the subproblem.

1. The subproblem. Let $MinCoins(i)$ be the minimum number of coins needed to represent the amount i.

Next, we think about the recursive relationship between subproblems. You notice that in order to make i, you could either have made $i-1$ and added a 1-coin, or made $i-3$ and added a 3-coin, or made $i-4$ and added a 4-coin. Using this observation, fill in the formal recursive definition for $MinCoins(i)$ below.

2. The recursive definition. The recursive relationship can be expressed as: (fill in both recursive case(s) and base case(s). You may not need every line provided!)

$$MinCoins(i) = \begin{cases} _____ & if\ i _____ \\ _____ & if\ i _____ \\ _____ & if\ i _____ \\ _____ & if\ i _____ \\ _____ & if\ i _____ \end{cases}$$

70. (10 points) In this problem, you will need to write algorithm pseudocode (step four of our four steps for writing dynamic programming algorithms) for the following problem.

The problem Having spied some snow on the mountains, you are thinking about ski season. You would like to choose a ski jumping course that allows for the maximum total distance in the air. The input to your problem is a set of n jumps at positions $s_1, s_2, \ldots, s_n$ on the slope along with jump distances $d_1, d_2, \ldots, d_n$ which indicate how far each jump will send you. Some jumps may launch you over other jumps, but you can assume that you can always land safely and complete the next jump. Let p_i be the farthest along jump that does not conflict with jump i (i.e., the farthest along jump you could have taken and still landed safely before jump i). If no such jump exists, let $p_i = 0$.

To solve this problem, we first write the English definition of the subproblems.

1. The subproblem. Let $MaxAir(i)$ be the maximum distance you can jump, assuming you only consider jumps 1 through i.

Next, we think about the recursive relationship between subproblems. You notice that if you take the ith jump, you get d_i distance, but you can only use jumps 1 through p_i. If you don't take the ith jump, then you can use jumps 1 through $i-1$. Using this observation, you write the formal recursive definition for $MaxAir(i)$ as follows.

2. The recursive definition.

$$MaxAir(i) = \begin{cases} 0 & if\, i = n \\ max(d_i + MaxAir(p_i), MaxAir(i-1)) & if\, i < 0 \end{cases}$$

3. How to memoize. We can use an array of length n to store the values of $MaxAir(i)$ and fill it in increasing order.

4. Write the algorithm. Using the above information, finish the pseudocode to find the maximum amount of air you can get on the ski jumps, given $s[1..n], d[1..n]$, and $p[1..n]$.

```
SkiJump(s[1..n], d[1..n], p[1..n]):
  Let array A be an array of length n
  # do we have any base cases?




  # how should we fill in the rest of the array?



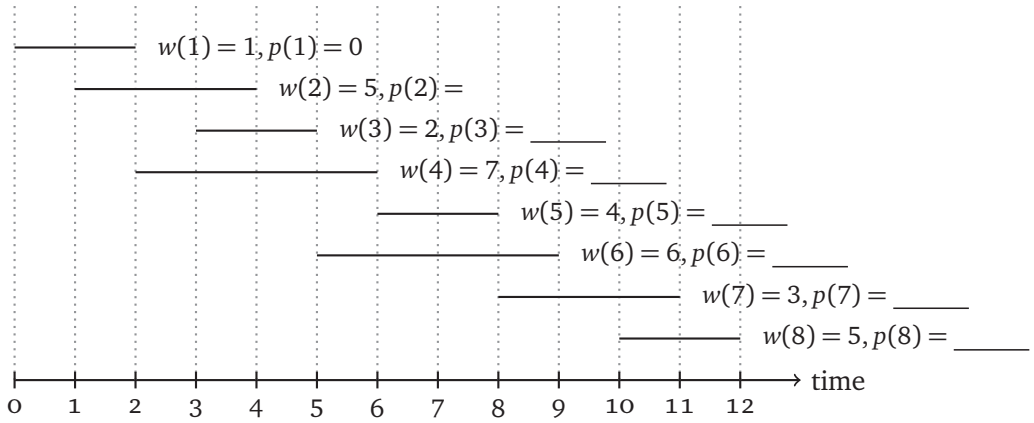

  # what should we return to get the final answer?
```

From Fall 2025 Exam 2 Practice 1

Recall the *weighted interval scheduling problem* from class. In this problem, we are given n intervals, each with a start time $s(i)$, a finish time $f(i)$, and a weight $w(i)$. The goal is to find a set of mutually compatible intervals with maximum total weight.

We will also define $p(i)$ to be the index of the last interval that finishes before interval i starts (or 0 if there is no such interval).

71. For the example below, fill in the values of $p(2)$ through $p(8)$.



In order to give a dynamic programming solution to this problem, we follow the four steps:

72. English description of the subproblems:
Let $MaxWeight(i)$ be the _____
_____.

73. You notice that the maximum weight of any set of compatible intervals at index i either:

- does not include interval i, so the maximum weight is just the maximum weight up to $i-1$, or
- does include interval i, so the maximum weight is the weight of interval i plus the maximum weight up to $p(i)$.

Using this observation, fill in the recursive part of the recursive definition of the subproblem with the notation provided for w and p values.

$$MaxWeight(i) = \begin{cases} 0 & if\ i = 0 \\ _____ & if\ i > 0 \end{cases}$$

74. To memoize this algorithm, we will use an array A of length _____ (be careful! only exact answers will get full credit) to store the needed values of $MaxWeight(i)$ and fill it in increasing order.

75. Finally, we write the pseudocode for the algorithm using the above information. Use an array A to store the values of $MaxWeight(i)$. Assume that your inputs are sorted by finishing time already.

```
MaxIntervalWeight(s[1..n], f[1..n], w[1..n], p[1..n]):
  # initialize an array A



  # do we have any base cases?



  # how should we fill in the rest of the array?



  # what should we return to get the final answer?
```

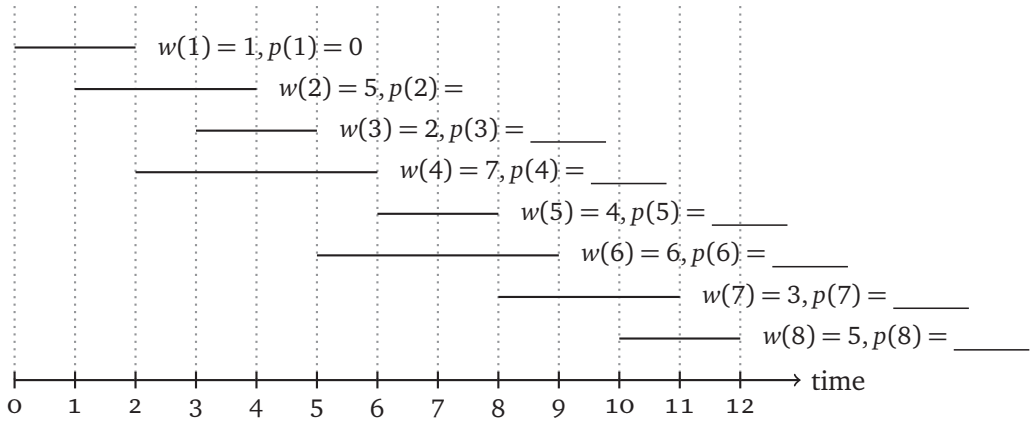
76. What are the values of A after running MaxIntervalWeight on the example above?

From Fall 2025 Exam 2 Practice 2

Recall the *weighted interval scheduling problem* from class. In this problem, we are given n intervals, each with a start time $s(i)$, a finish time $f(i)$, and a weight $w(i)$. The goal is to find a set of mutually compatible intervals with maximum total weight.

We will also define $p(i)$ to be the index of the last interval that finishes before interval i starts (or 0 if there is no such interval).

77. For the example below, fill in the values of $p(3)$ through $p(8)$.



In order to give a dynamic programming solution to this problem, we follow the four steps:

78. English description of the subproblems:
Let $MaxWeight(i)$ be the _____
_____.

79. You notice that the maximum weight of any set of compatible intervals at index i either:

- does not include interval i, so the maximum weight is just the maximum weight up to $i-1$, or
- does include interval i, so the maximum weight is the weight of interval i plus the maximum weight up to $p(i)$.

Using this observation, fill in the recursive part of the recursive definition of the subproblem with the notation provided for w and p values.

$$MaxWeight(i) = \begin{cases} 0 & if\ i = 0 \\ _____ & if\ i > 0 \end{cases}$$

80. To memoize this algorithm, we will use an array A of length _____ (be careful! only exact answers will get full credit) to store the needed values of $MaxWeight(i)$ and fill it in increasing order.

81. Finally, we write the pseudocode for the algorithm using the above information. Use an array A to store the values of $MaxWeight(i)$. Assume that your inputs are sorted by finishing time already.

```
MaxIntervalWeight(s[1..n], f[1..n], w[1..n], p[1..n]):
  # initialize an array A



  # do we have any base cases?




  # how should we fill in the rest of the array?









  # what should we return to get the final answer?
```

82. What are the values of A after running MaxIntervalWeight on the example above?

From Fall 2025 Final Exam

The next three questions relate to the following problem:

In preparation for the University of Montana CS department's annual winter bowling party (6-7:30pm this Saturday, December 13th, 2025 at Westside Lanes), you have been practicing with the following, simpler version of bowling. There are n pins in a single row, numbered from 1 to n. Each pin i has a point value $P[i]$ associated with it. You can knock down either one single pin or two adjacent pins each time you roll the ball. Because you have been practicing so much, you always hit the pins you aim for. If you knock down pin i, you earn $P[i]$ points. If you knock down pins i and $i+1$, you earn the product $P[i] \cdot P[i+1]$ points.

You want to maximize your score by choosing which pins to knock down.

For example, if you have 4 pins with values $P = [2, 3, 5, 1]$, one optimal strategy is to knock down pin 1 alone, pins 2 and 3 together, and pin 4 alone, earning $2 + (3 \cdot 5) + 1 = 18$ points.

83. If you formulate subproblems for this bowling problem as *Let $MaxPoints(i)$ be the maximum points you can earn by knocking down pins from i to n*, how will you compute your final answer?

    A. $MaxPoints(1)$
    B. $MaxPoints(n)$
    C. $MaxPoints(MaxPoints)$
    D. $MaxPoints(1, n)$
    E. $MaxPoints(n, n)$

84. If you formulate subproblems for this bowling problem as *Let $MaxPoints(i)$ be the maximum points you can earn by knocking down pins from 1 to i*, how will you compute your final answer?

    A. $MaxPoints(1)$
    B. $MaxPoints(n)$
    C. $MaxPoints(MaxPoints)$
    D. $MaxPoints(1, n)$
    E. $MaxPoints(n, n)$

85. If you formulate subproblems for this bowling problem as *Let $MaxPoints(i)$ be the maximum points you can earn by knocking down pins from 1 to i*, which of the following is a correct recurrence relation for $MaxPoints(i)$?

    A. $MaxPoints(i) = \max\{P[i] + MaxPoints(i-1), P[i] \cdot P[i+1] + MaxPoints(i+2)\}$
    B. $MaxPoints(i) = \max\{P[i] + MaxPoints(i-1), P[i] \cdot P[i-1] + MaxPoints(i-2)\}$
    C. $MaxPoints(i) = \max\{MaxPoints(i-1), MaxPoints(i-2)\}$
    D. $MaxPoints(i) = \max\{P[i], P[i] \cdot P[i-1]\}$
    E. None of the above